

Machine Learning Algorithms Laboratory

Experiment 4 Report

Experiment No.	4
Experiment Title	Binary Classification using Linear and Kernel-Based Models 1
Student Name	Gautham S
Register Number	3122247001016
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	

1 Objective

The objective of this experiment is to classify emails as spam or ham using Logistic Regression and Support Vector Machine (SVM) classifiers, and to study how hyperparameter tuning affects the classification performance of both models.

2 Theory

2.1 Logistic Regression

Logistic Regression is a probabilistic classifier that models the probability of a sample belonging to class 1 using the sigmoid function:

$$P(y = 1|x) = \frac{1}{1 + e^{-(w^T x + b)}} \quad (1)$$

A threshold of 0.5 is generally used to convert this probability into a class label. Regularization can be added to prevent overfitting: L1 regularization can shrink some coefficients exactly to zero (useful for feature selection), while L2 regularization keeps all features but penalizes large weights.

2.2 Support Vector Machine

SVM is a margin based classifier that tries to find the hyperplane which best separates the two classes by maximizing the margin between them. Only the points closest to the boundary (support vectors) actually decide where the boundary goes. SVM can also use kernels (linear, polynomial, RBF, sigmoid) to handle data that is not linearly separable by mapping it into a higher dimension.

3 Dataset Description

Dataset	Description
---------	-------------

Dataset Name	Spambase Dataset
Dataset Source	Kaggle https://www.kaggle.com/datasets/somesh24/spambase
Number of Samples	4601
Number of Features	57 numerical features
Number of Classes	2 (0 = Ham, 1 = Spam)
Missing Values	None
Class Distribution	Ham: 2788 (60.6%), Spam: 1813 (39.4%)
Train-Test Split	80% Train (3680 samples), 20% Test (921 samples)

4 Exploratory Data Analysis

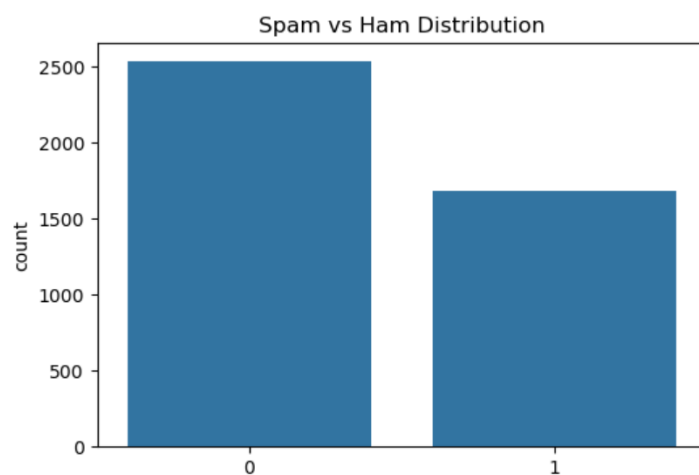


Figure 1: Spam vs Ham Distribution

Inference: The dataset is moderately imbalanced, with about 61% ham and 39% spam emails. This is not extreme, but it is still a good idea to look at precision, recall and F1-score along with accuracy instead of relying on accuracy alone.

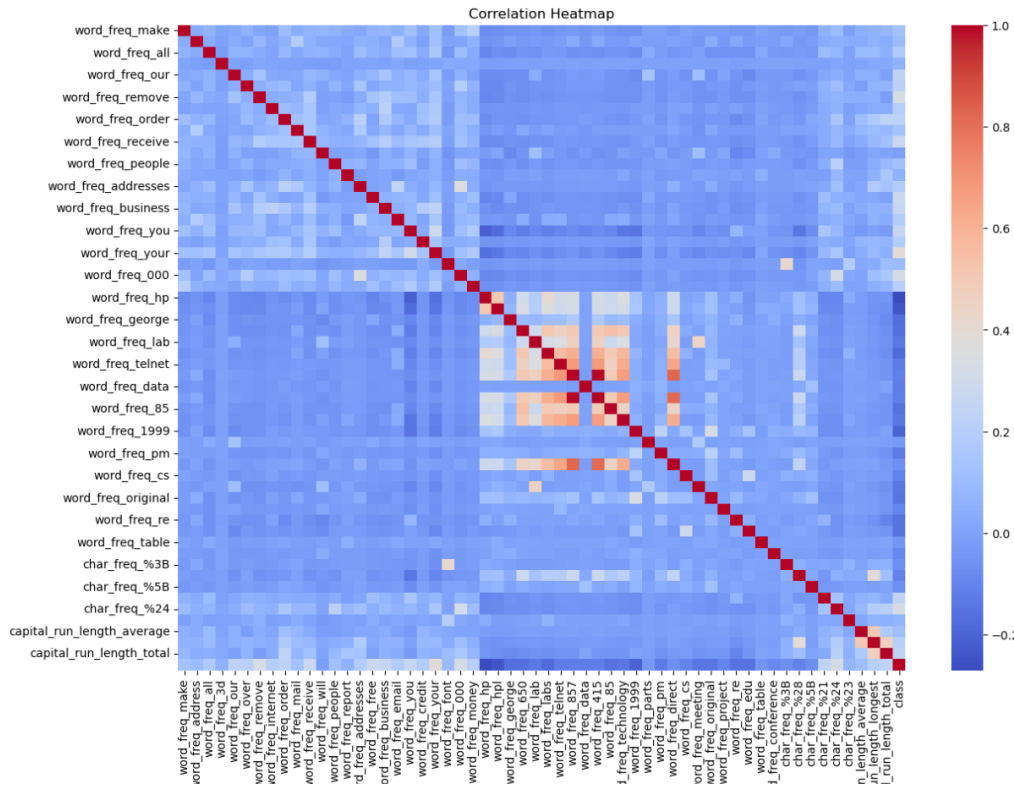


Figure 2: Correlation Heatmap of Spambase Features

Inference: Most of the word frequency features do not show strong correlation with each other. A few of the capital run length features show mild correlation among themselves, which is expected since they are all derived from the same kind of information (usage of capital letters).

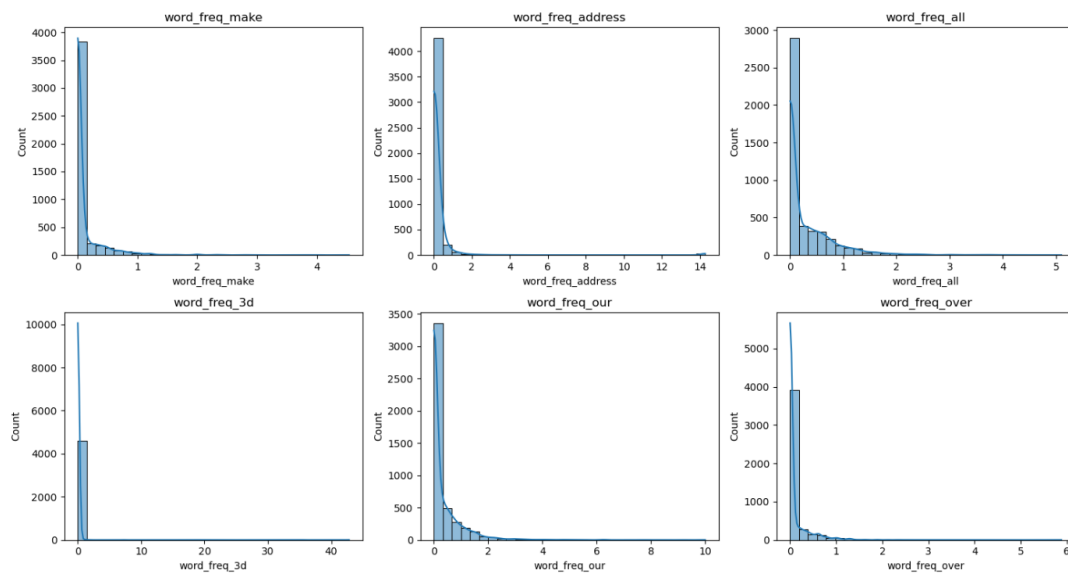


Figure 3: Histograms of the First 6 Features

Inference: Most of the word frequency features are heavily right skewed, with a large number of zero values and a long tail. This is expected since a particular word will not

appear in every single email.

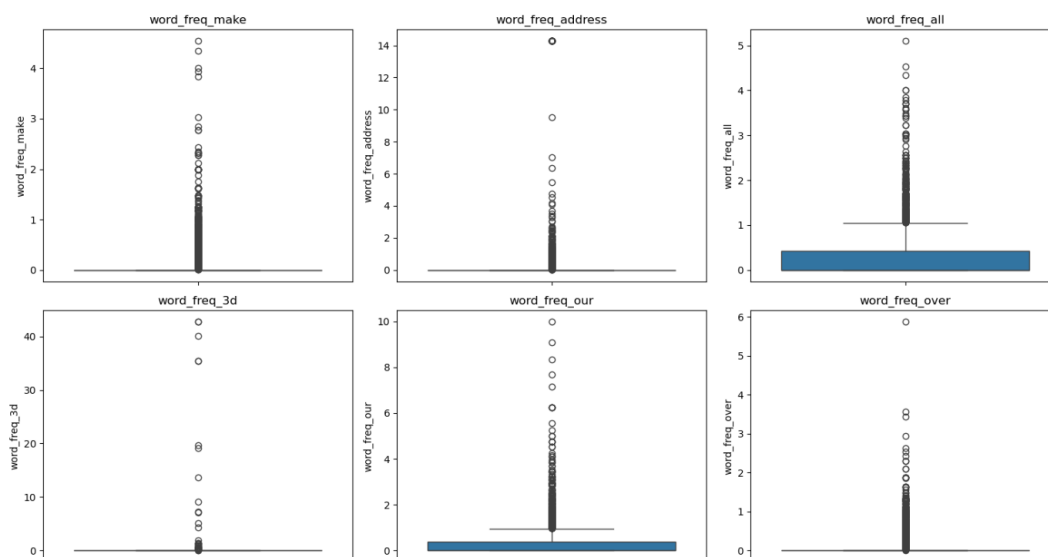


Figure 4: Boxplots of the First 6 Features

Inference: The boxplots confirm the skew seen in the histograms, most values are close to zero with a number of high value outliers. This is fairly normal for word frequency type data.

5 Data Preprocessing

The Spambase dataset had no missing values, so no imputation was required. The features were split into X (57 word/character frequency features) and y (class label). The data was split into 80% training and 20% testing using stratified sampling to preserve the class ratio. Since both Logistic Regression and SVM are sensitive to feature scale, all features were standardized using `StandardScaler`.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

6 Model Training and Evaluation

6.1 Logistic Regression

A baseline Logistic Regression model was trained first with default parameters.

```
log_reg = LogisticRegression(random_state=42)
log_reg.fit(X_train_scaled, y_train)
```

The baseline model gave an accuracy of 0.9294, precision of 0.9209, recall of 0.8981 and F1-score of 0.9093, training in about 0.0107 seconds.

Hyperparameters were then tuned using GridSearchCV over penalty (L1/L2), C and solver.

```
param_grid_lr = {
    'penalty': ['l1', 'l2'],
    'C': [0.01, 0.1, 1, 10, 100],
    'solver': ['liblinear', 'saga']
}

grid_lr = GridSearchCV(LogisticRegression(max_iter=5000),
    param_grid_lr,
                        cv=5, scoring='accuracy', n_jobs=-1)
grid_lr.fit(X_train_scaled, y_train)
```

Metric	Value
Accuracy	0.9305
Precision	0.9211
Recall	0.9008
F1 Score	0.9109
Training Time (s)	23.9927

Table 2: Tuned Logistic Regression Performance (C=1, penalty=l2, solver=saga)

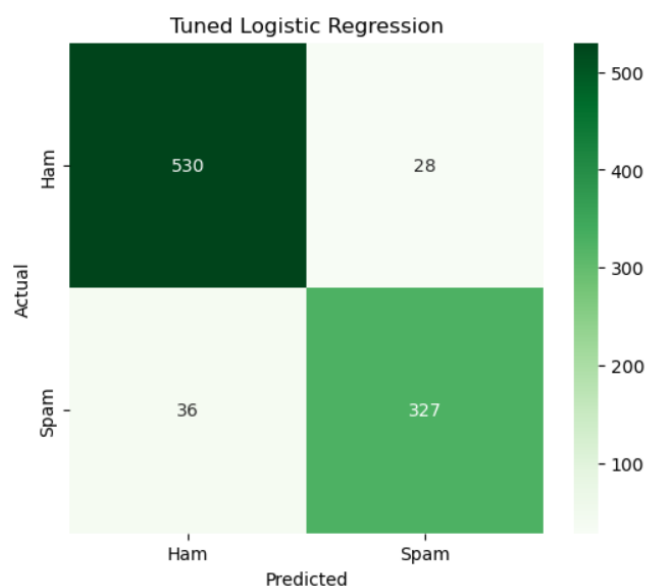


Figure 5: Confusion Matrix - Tuned Logistic Regression

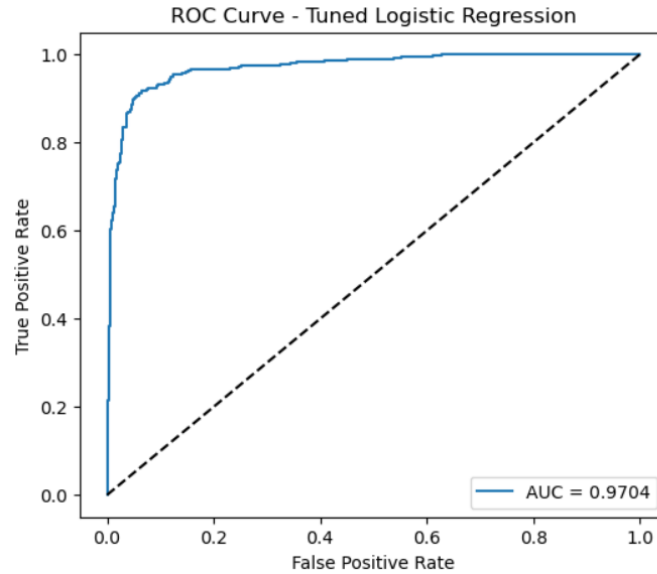


Figure 6: ROC Curve - Tuned Logistic Regression

Inference: Tuning gave only a very small improvement in test accuracy (0.9294 to 0.9305). The best parameters chosen were L2 regularization with $C=1$, which is basically a moderate regularization strength, meaning the untuned default was already close to optimal for this dataset. The ROC-AUC was high (around 0.97), showing the model separates the two classes well.

6.2 Support Vector Machine

SVM was first trained with four different kernels to compare their behaviour.

```
kernels = ['linear', 'poly', 'rbf', 'sigmoid']

for kernel in kernels:
    svm = SVC(kernel=kernel, random_state=42)
    svm.fit(X_train_scaled, y_train)
    y_pred = svm.predict(X_test_scaled)
    # accuracy, f1 calculated here
```

Kernel	Accuracy	F1 Score	Training Time (s)
Linear	0.9294	0.9093	0.2370
Polynomial	0.7796	0.6220	0.2315
RBF	0.9273	0.9055	0.1295
Sigmoid	0.8849	0.8528	0.1340

Table 3: SVM Kernel-wise Performance (default parameters)

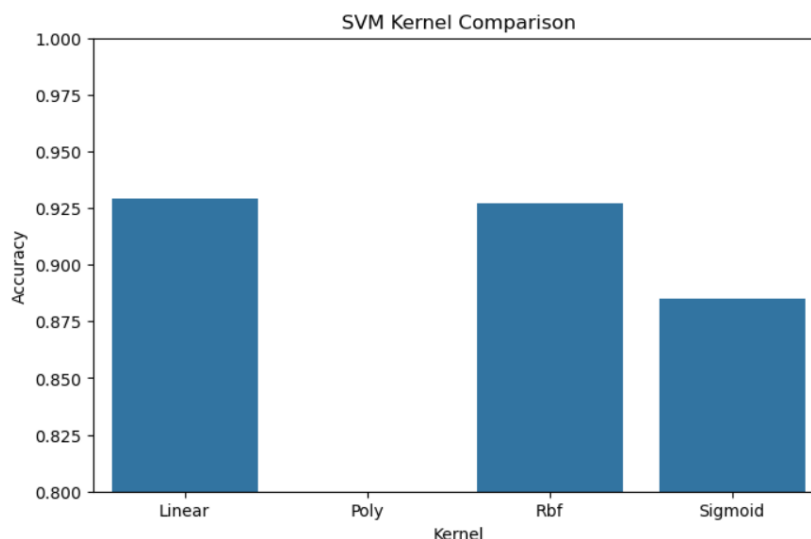


Figure 7: SVM Kernel Comparison (Accuracy)

Inference: Linear and RBF kernels perform the best and are quite close to each other, while the Polynomial kernel performs noticeably worse (accuracy dropped to 0.78). This suggests that the spam/ham data is close to being linearly separable once standardized, so the extra complexity of a polynomial boundary does not help and might even overfit to less useful patterns.

SVM hyperparameters were then tuned using GridSearchCV over kernel, C, gamma and degree (for polynomial).

```
param_grid_svm = [
    {'kernel': ['linear'], 'C': [0.1, 1, 10, 100]},
    {'kernel': ['rbf', 'sigmoid'], 'C': [0.1, 1, 10, 100], 'gamma': ['scale', 'auto']},
    {'kernel': ['poly'], 'C': [0.1, 1, 10, 100], 'gamma': ['scale', 'auto'], 'degree': [2, 3, 4]}
]

grid_svm = GridSearchCV(SVC(), param_grid_svm, cv=5, scoring='accuracy', n_jobs=-1)
grid_svm.fit(X_train_scaled, y_train)
```

GridSearchCV picked {C=10, gamma=scale, kernel=rbf} as the best combination, with a best CV accuracy of 0.9332.

Metric	Value
Accuracy	0.9207
Precision	0.9143
Recall	0.8815
F1 Score	0.8976
Training Time (s)	0.1130

Table 4: Tuned SVM Performance (C=10, gamma=scale, kernel=rbf)

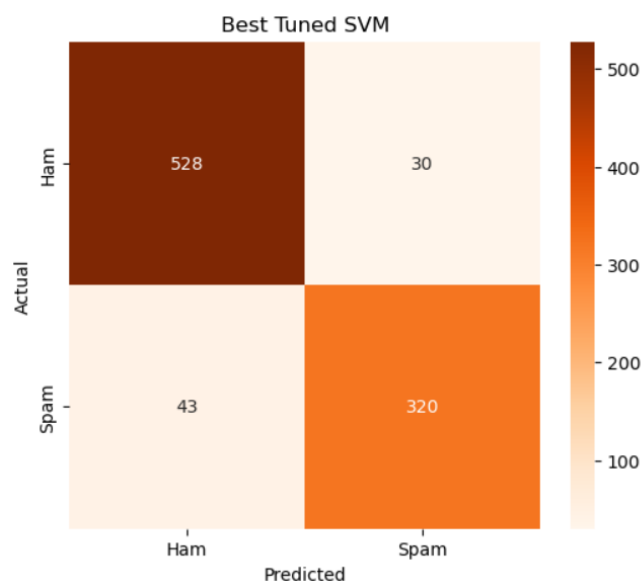


Figure 8: Confusion Matrix - Tuned SVM (RBF)

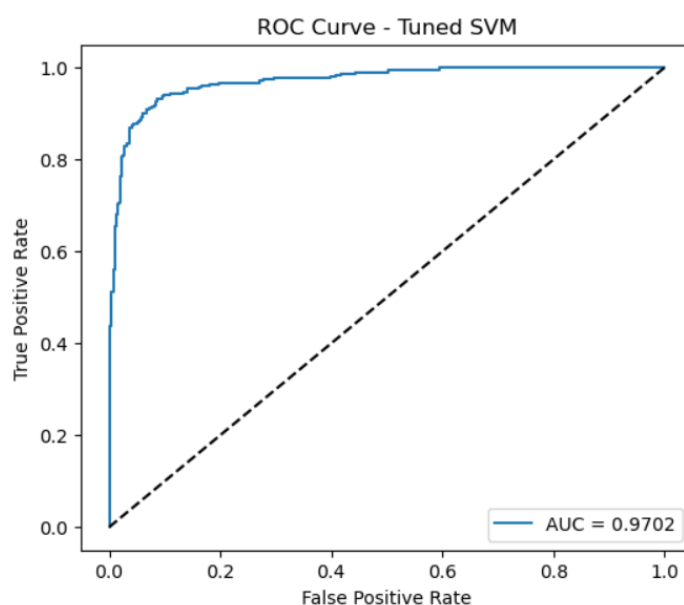


Figure 9: ROC Curve - Tuned SVM (RBF)

Inference: Interestingly, the tuned RBF kernel had the best cross-validation accuracy (0.9332) during grid search, but on the actual held-out test set it scored slightly lower (0.9207) than the plain default Linear kernel from the earlier comparison (0.9294). This can happen because the CV accuracy is only an average estimate over the training folds and does not guarantee the best result on one particular test split. It is a good reminder that the model with the best cross-validation score is not always the best model on every single test set.

7 Hyperparameter Tuning Results

Model	Search Method	Best Parameters	Best CV Accuracy
Logistic Regression	Grid Search	C=1, penalty=l2, solver=saga	0.9242
SVM	Grid Search	C=10, gamma=scale, kernel=rbf	0.9332

Table 5: Hyperparameter Tuning Summary

8 K-Fold Cross-Validation (K=5)

Fold	Logistic Regression	SVM
Fold 1	0.9375	0.9443
Fold 2	0.9253	0.9416
Fold 3	0.9307	0.9348
Fold 4	0.9076	0.9307
Fold 5	0.9171	0.9266
Average	0.9236	0.9356

Table 6: 5-Fold Cross Validation Results

Inference: During cross-validation on the training set, the tuned SVM is consistently better than Logistic Regression across all 5 folds, with a higher average accuracy (0.9356 vs 0.9236). This is a bit different from what was seen on the single test split above, which shows why cross-validation is generally considered a more reliable estimate of a model's performance than a single train-test split.

9 Comparative Analysis

Criterion	Logistic Regression	SVM
Accuracy (Test)	0.9305	0.9207
Model Complexity	Low	High
Training Time	Low	High
Interpretability	High	Low

Table 7: Comparative Analysis of Logistic Regression and SVM

10 Observations

- **Best-performing classifier:** Based on the single test split, tuned Logistic Regression gave the highest accuracy (0.9305). But based on 5-fold cross validation (a more reliable estimate), SVM performed better on average (0.9356 vs 0.9236). So it depends on which evaluation is trusted more, but SVM is arguably the more robust choice here.

- **Impact of regularization:** For Logistic Regression, GridSearchCV chose L2 regularization with a moderate C value ($C=1$), and the improvement over the unregularized default was very small. This suggests the baseline model was not overfitting significantly to begin with, so regularization only provided a minor benefit here.
- **Kernel behaviour in SVM:** The linear and RBF kernels performed the best and were close to each other, showing that the spambase data is close to linearly separable once scaled. The polynomial kernel performed noticeably worse, likely because a higher degree polynomial decision boundary is too flexible for this kind of frequency based data and does not generalize as well.
- **Bias-variance trade-off:** Logistic Regression, being a simpler linear model, tends to have higher bias and lower variance, it is more stable but might miss more complex patterns. SVM with the RBF kernel can model more complex, non-linear boundaries (lower bias) but is more sensitive to the choice of hyperparameters like C and gamma (higher variance), which is possibly why its test set accuracy dropped slightly compared to its cross-validation accuracy.

11 Conclusion

In this experiment, Logistic Regression and SVM were used to classify emails as spam or ham. Logistic Regression gave a slightly better result on the fixed test split after tuning (0.9305 accuracy), while SVM (RBF kernel) had a higher average cross-validation accuracy (0.9356), making the comparison not entirely one sided. Logistic Regression is simpler, trains much faster, and is more interpretable since its coefficients directly show feature importance, while SVM (especially with kernels) is more computationally expensive and harder to interpret but can capture more complex boundaries. For this particular spam classification problem, since the data is close to linearly separable, Logistic Regression is a good practical choice considering it gives comparable performance with much less training time and better interpretability.

12 References

1. Scikit-learn Documentation: Logistic Regression. https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
2. Scikit-learn Documentation: Support Vector Machines. <https://scikit-learn.org/stable/modules/svm.html>
3. Scikit-learn Documentation: Hyperparameter Optimization. https://scikit-learn.org/stable/modules/grid_search.html
4. Spambase Dataset, Kaggle. <https://www.kaggle.com/datasets/somesh24/spambase>